Sistemas Distribuidos: Tolerancia a Fallas

CCPD

Universidad Central de Venezuela
Facultad de Ciencias
Postgrado en Computación

Abril – Julio 2005



Objetivos

- Comprender y describir el problema de desarrollar aplicaciones distribuidas tolerantes a fallas.
- Comprender y establecer diferencias entre los conceptos de algoritmo robusto y algoritmo estable
- Estudiar ejemplos de aplicaciones robustas y estables.

Tolerancia a fallas

Las interrupciones causadas debido a fallas pueden ser graves en muchos casos, tales como:

- procesamiento de transacciones en línea,
- control de procesos,
- comunicaciones de una comunidad basadas en el computador, etc.

Para evitar las interrupciones debido a fallas y asegurar la disponibilidad, algunos sistemas son diseñados para ser tolerantes a fallas.

Aspectos Relacionados con Tolerancia a Fallas

- **Disponibilidad: (Availability)** Se refiere a la propiedad de que el sistema esté operando correctamente en cualquier instante de tiempo.
- **Confiabilidad: (Reliability)** Propiedad de que el sistema pueda estar ejecutándose de forma continua, sin fallas.

  Ejemplo: Un sistema "no operativo" un milisegundo cada hora tiene una disponibilidad por encima del 99,999%, pero es poco "confiable". Por el contrario, un sistema que no falla nunca, pero que es "tumbado" dos semanas durante un mes es altamente confiable pero solo 96% disponible

Aspectos Relacionados con Tolerancia a Fallas

- **Seguridad: (Safety)** Se refiere a que un sistema cuando falla temporalmente no ocasiona situaciones catastróficas.

- **Fácil Mantenimiento : (Maintainability)** Se refiere a que tan fácil es reparar un sistema que ha fallado.

- Security (Seguridad)

Aspectos Relacionados con Tolerancia a Fallas

- Un sistema falla cuando no puede satisfacer los requerimientos para los cuales fué diseñado, esto es cuando no pueden ser provistos (completamente) uno ó mas de sus servicios.

- Entonces: el aspecto más importante de la Tolerancia a Fallas es que el sistema pueda proveer el servicio aún en presencia de fallas.

Modelos de fallas

1. Falla de procesos
2. Falla de máquinas: Todos los procesos en esa máquina mueren. En lo que concierne a un proceso en particular es similar al caso muerte de un proceso.
3. Falla de la red: puede particionar una red en sub-redes haciendo imposible a un proceso comunicarse con otro en una subred diferente

Modelos de fallas de procesos

1. Crash falla o muerte del proceso: Un proceso detiene su funcionamiento y jamás reasume su ejecución
2. Falla de omisión: El proceso omite enviar algunos mensajes a algunos procesos
3. Falla maliciosa o **Comportamiento Bizantino**: El proceso tiene un comportamiento aleatorio y arbitrario. Son muy amplios en su naturaleza,por lo que muchas fallas son modeladas como comportamiento Bizantino.

Algoritmos robustos y Algoritmos estables

Algoritmo Robusto: Protegen contra *fallas permanentes* de ciertos componentes (Procesos).

Los procesos sobrevivientes mantienen un comportamiento correcto durante la reparación y reconfiguración del sistema.

- Deben ser usados cuando no se puede interrumpir un servicio.
- Ejemplo: Algoritmo de Lamport *y otros* para resolver el problema del comportamiento bizantino



Algoritmos robustos y Algoritmos estables

Algoritmo Estable: Ofrecen protección contra fallas transitorias (conducta temporal errónea) de algunos componentes (Procesos) del sistema. Cuando las condiciones retornan a la normalidad, el proceso reasume su ejecución, pero debido a la falla temporal el estado global (o configuración) podría ser arbitrario o incoherente durante un lapso de tiempo.

La estabilidad garantiza la convergencia al comportamiento requerido y a un estado global válido en un tiempo finito.

- Ejemplo: Algoritmo de Dijkstra para exclusión mutua



Algoritmos Robustos

Son algoritmos diseñados para garantizar que un sistema siga operando aún cuando durante su ejecución algunos de sus procesos componentes puedan fallar.

Ejemplo: Acuerdo Bizantino

- En ocasiones cuando un sistema falla su funcionamiento es arbitrario, y se puede enviar información errónea.
- Con el acuerdo Bizantino se busca alcanzar un acuerdo sobre un determinado valor, en un sistema que trabaja con algunos procesos defectuosos.
- En el problema del acuerdo Bizantino un proceso elegido arbitrariamente, llamado el origen, envía un valor inicial al resto de los procesos.

Ejemplo: Acuerdo Bizantino

Una solución al problema del acuerdo Bizantino debe garantizar que:

- Todos lo procesos no defectuosos deben estar de acuerdo con el mismo valor.
- Si el proceso origen es no defectuoso, entonces el valor convenido por los procesos no defectuosos debe ser el mismo valor del proceso origen.

Algoritmo de Lamport-Shostak-Pease

- Se conoce como OM(m) (Oral Messages Algorithm)
- Es un algoritmo recursivo: Divide al sistema en subsistemas cada vez más pequeños.
- El algoritmo OM(m) resuelve el problema para un número de procesos defectuosos acotado por $\mathbf{n} \geq 3\mathbf{m}+1$.
  - $\mathbf{m} \geq 0$, es la cantidad de procesos defectuosos.
  - **n** cantidad de procesos del sistema.

Algoritmo Lamport-Shostak-Pease

- OM(m) m>0:
  - El proceso origen genera su valor y lo envía a los n-1 procesos restantes.
  - Cada proceso i, guarda el valor que recibe del proceso origen.
  - El proceso i actúa como nuevo proceso origen e inicializa OM(m-1,n-1) donde envía el valor v_i a los restantes n-2 procesos
  - El proceso i escoge el valor más repetido entre los valores v_j ($i \neq j$) recibidos de los procesos j

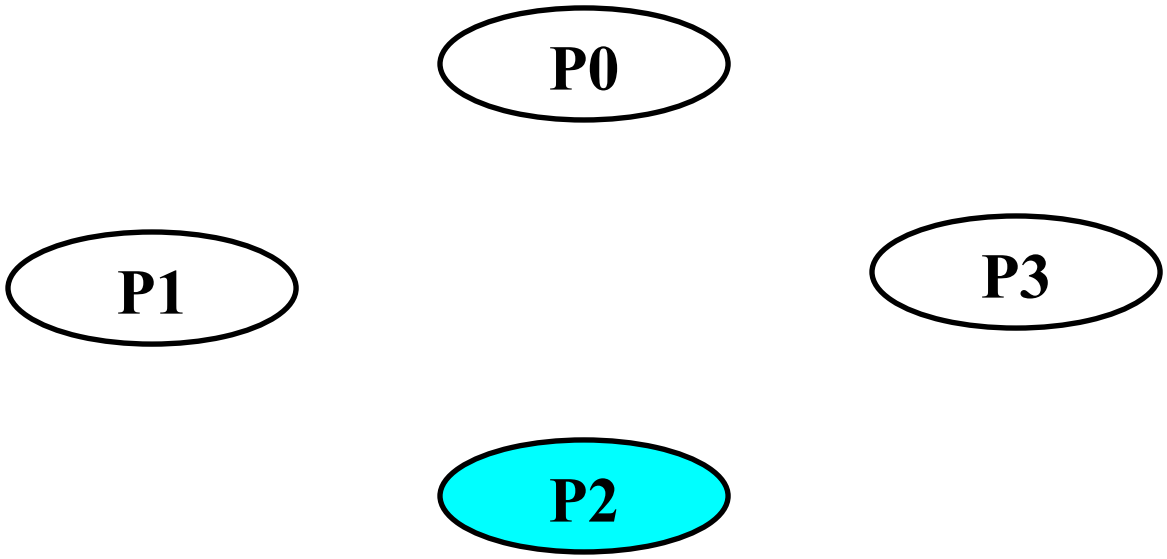
Algoritmo Lamport-Shostak-Pease

- OM(m) m=0:
  - El proceso Origen envía su valor a los restantes procesos
  - Cada proceso guarda el valor recibido desde el origen

Ejemplo 1

Se tienen 4 procesos, uno de ellos está defectuoso: m=1 , n=4. El proceso Origen no está defectuoso.

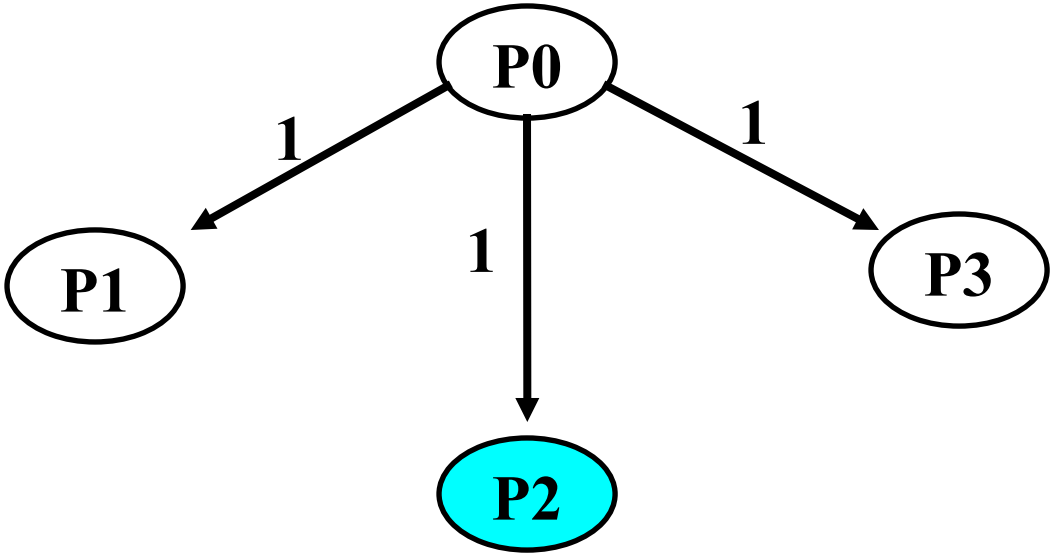
OM(1)



Ejemplo 1

m=1, n=4

- P0={}
- P1={1}
- P2={1}
- P3={1}

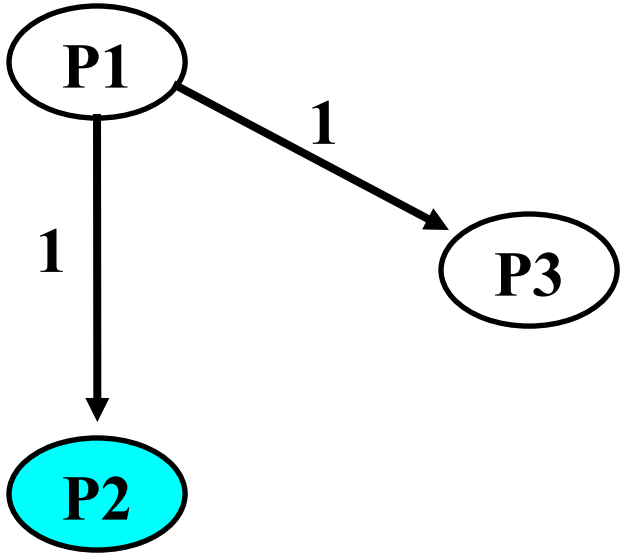


Se generan tres (3) ejecuciones separadas de OM(0)

Ejemplo 1

OM(0) m=0,n=3

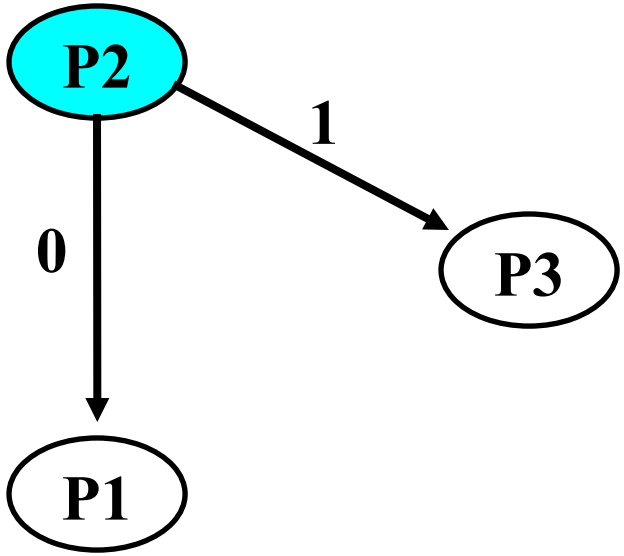
- P1={1}
- P2={1,1}
- P3={1,1}



Ejemplo 1

OM(0) m=0,n=3

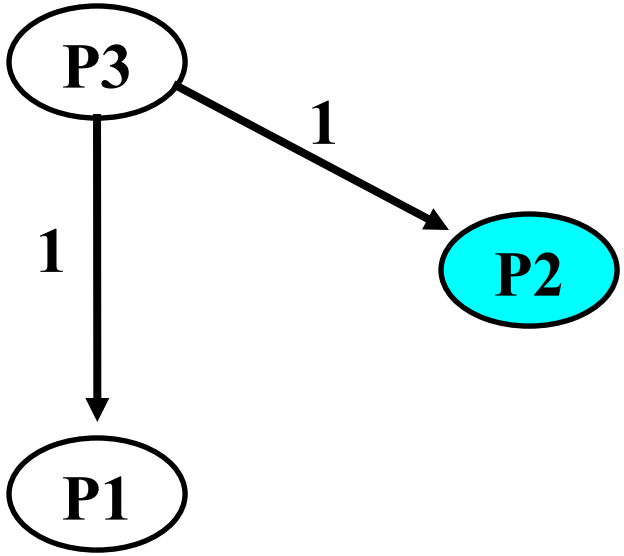
- P2={1,1}
- P1={1,0}
- P3={1,1,1}



Ejemplo 1

OM(0) m=0,n=3

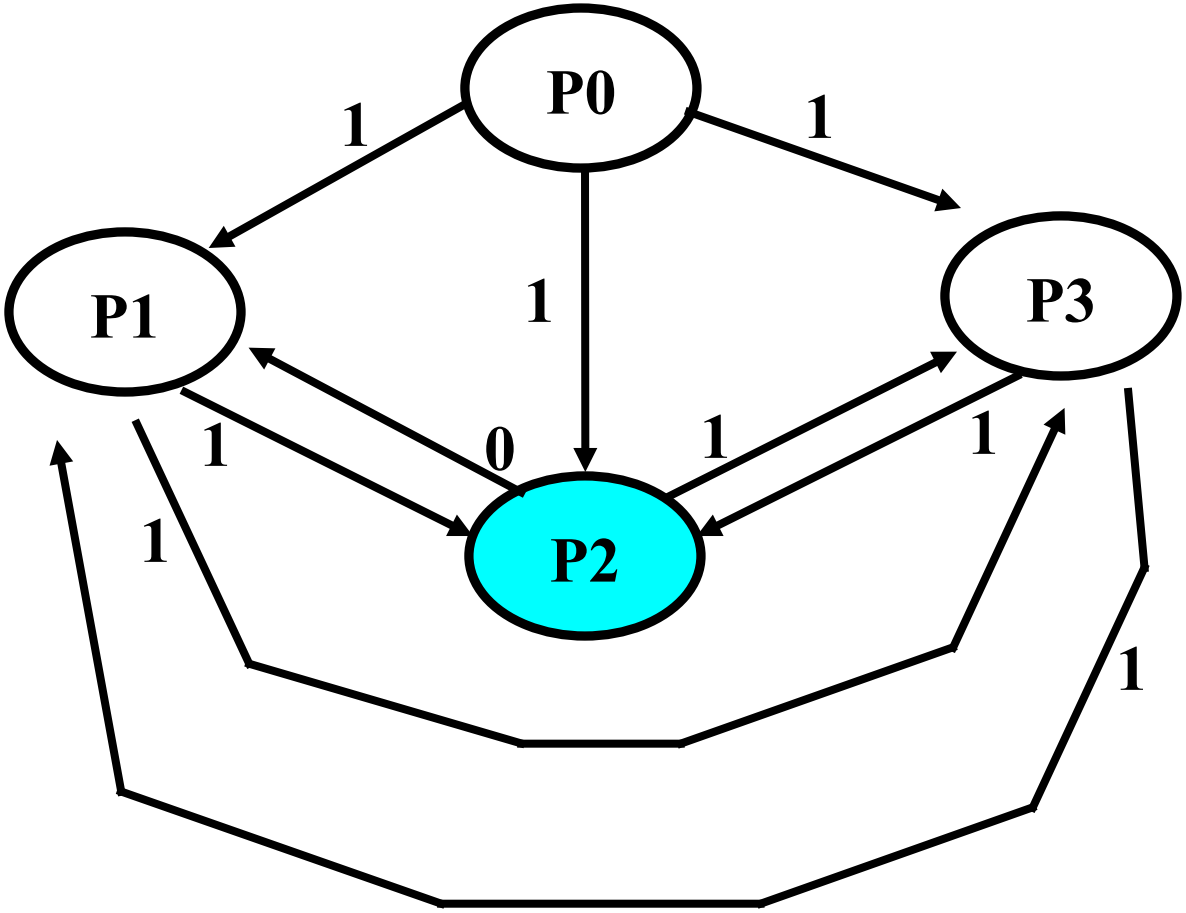
- P3={1,1,1}
- P1={1,0,1}
- P2={1,1,1}



Ejemplo 1

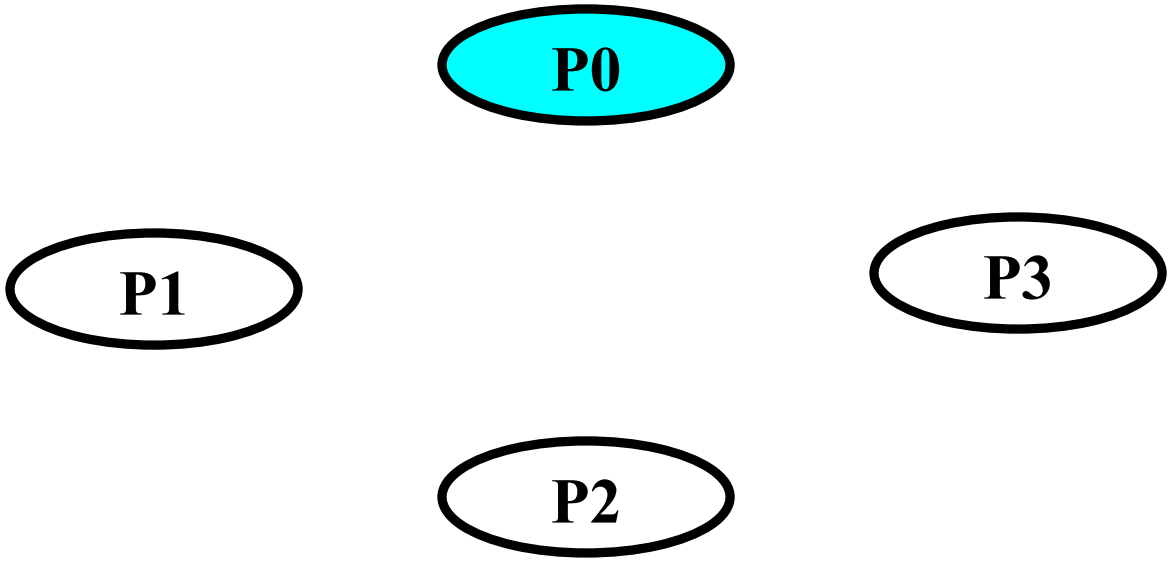
Estado final

- P3={1,1,1} =1
- P1={1,0,1} =1
- P2={1,1,1} = 1



Ejemplo 2

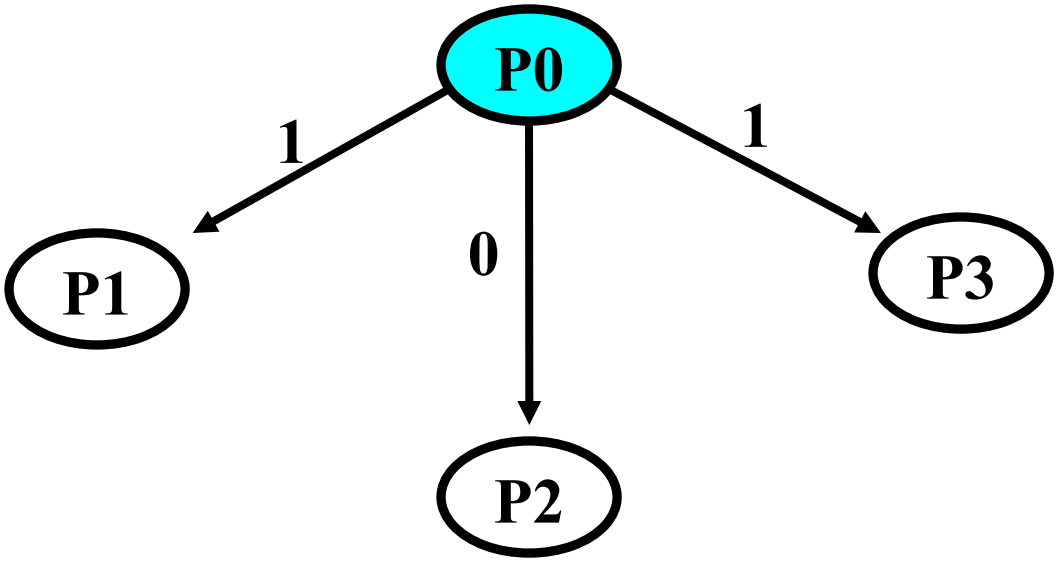
Se tienen 4 procesos y el inicial está defectuoso y envía datos erróneos m=1 , n=4.



Ejemplo 2

OM(1) m=1,N=4

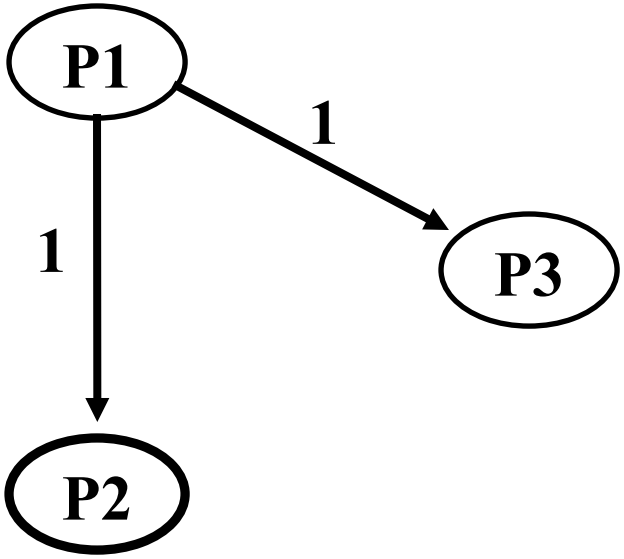
- P0={}
- P1={1}
- P2={0}
- P3={1}



Ejemplo 2

OM(0)

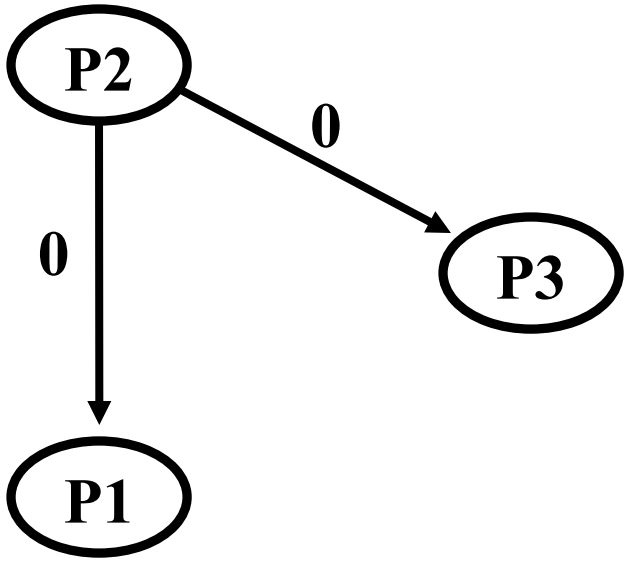
- P1={1}
- P2={0,1}
- P3={1,1}



Ejemplo 2

OM(0)

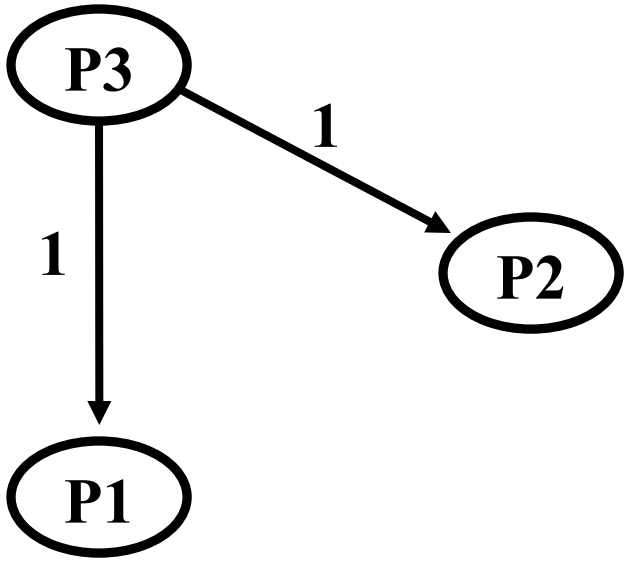
- P2={0,1}
- P1={1,0}
- P3={1,1,0}



Ejemplo 2

OM(0)

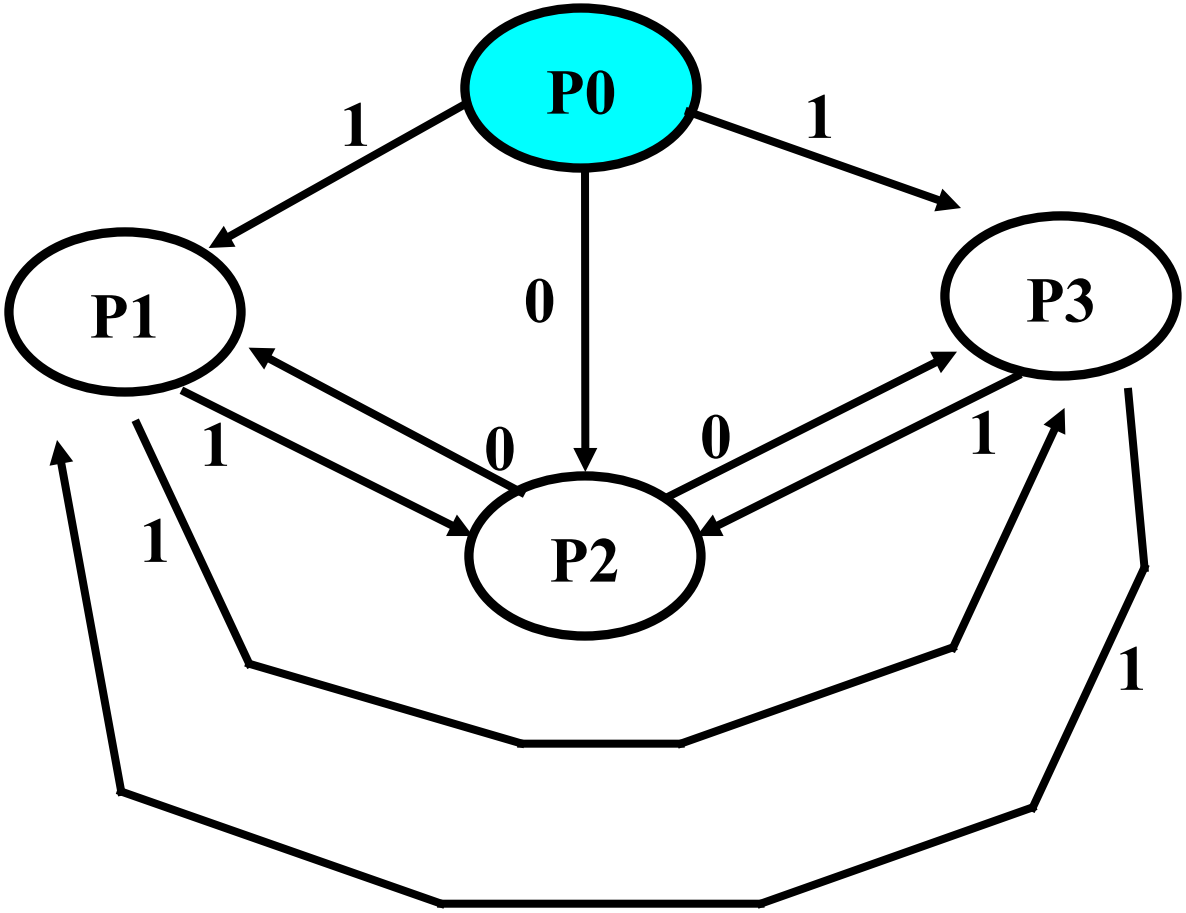
- P3={1,1,0}
- P1={1,0,1}
- P2={0,1,1}



Ejemplo 2

Estado final

- P3={1,0,1}=1
- P1={0,1,1}=1
- P2={1,1,0}=1



Conclusiones

- Algunas aplicaciones del acuerdo Bizantino son: Problemas de sincronización de reloj, atomicidad de transacciones y Sistemas tolerantes a fallas en general.
- Si el proceso origen está defectuoso, entonces los procesos no defectuosos pueden acordar cualquier valor, pero el mismo todos.
- Es irrelevante el valor acordado por los procesos defectuosos.



Algoritmos robustos y Algoritmos estables

Algoritmo Estable: Ofrecen protección contra fallas transitorias (conducta temporal errónea) de algunos componentes (Procesos) del sistema. Cuando las condiciones retornan a la normalidad, el proceso reasume su ejecución, pero debido a la falla temporal el estado global (o configuración) podría ser arbitrario o incoherente durante un lapso de tiempo.

La estabilidad garantiza la convergencia al comportamiento requerido y a un estado global válido en un tiempo finito.

- Ejemplo: Algoritmo de Dijkstra para exclusión mutua



Exclusión mutua

Cuando en un sistema o aplicación existen recursos distribuidos que pueden ser usados concurrentemente por múltiples procesos, es necesario garantizar que **sólo un proceso a la vez haga uso de dicho recurso** compartido a fin de resguardar la coherencia en su utilización.

Ejemplo: Actualización de un directorio, una impresora

Requerimientos para algoritmos de exclusión mutua

1. Debe garantizar que sólo un proceso a la vez acceda a la Sección Crítica
2. Debe ser libre de abrazo mortal
3. No debe dejar a algún proceso esperando indefinidamente por acceder a la SC
4. Los requerimientos deben ser satisfechos en el orden en que llegan al sistema
5. Debe ser tolerante a fallas, reconociendo la existencia de fallas y superándolas.

Clasificación de algoritmos para exclusión mutua en S.D.

- **Basados en tokens**: Como sólo un proceso puede entrar a la SC en un momento dado, el derecho a entrar es materializado por un objeto especial, único en el sistema, llamado *Token*
- **Basados en permiso**: Cuando un proceso quiere entrar en la sección crítica (SC) le pide permiso a los otros procesos y espera hasta que le sean concedido todos los permisos.

Algoritmos basados en Token

El aspecto a controlar aquí es el movimiento del *Token* de un proceso a otro, a fin de garantizar que toda solicitud sea satisfecha.

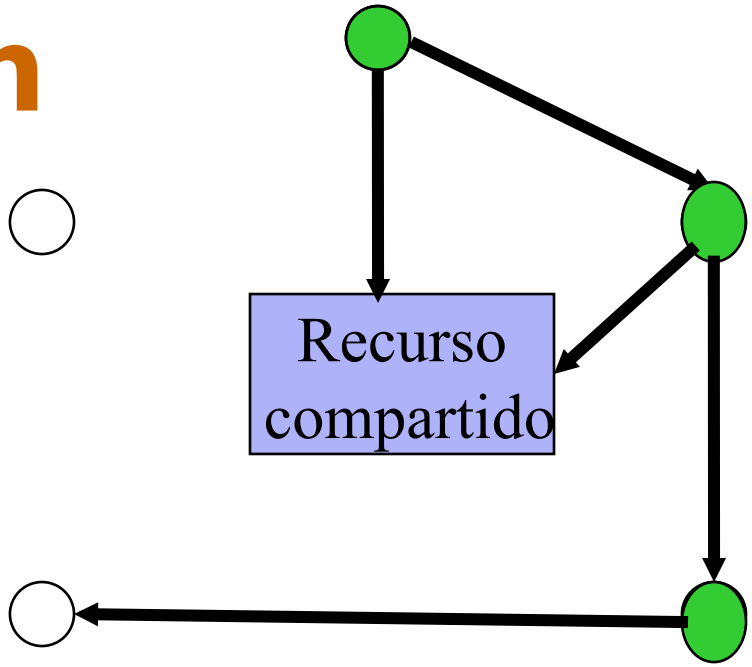
Hay dos posibilidades a considerar:

- Movimiento perpetuo
- Método de preguntar por el Token.

Algoritmos basados en movimiento perpetuo del Token

1. El token viaja desde un proceso a otro, para conceder el derecho de entrar a la SC
2. Para no ignorar los requerimientos de algún proceso, se considera una topología lógica de anillo dirigido, para efectos del movimiento del token
3. Si un proceso recibe el token y no está interesado en entrar a la SC, lo pasa al siguiente proceso en el anillo.

Algoritmos basados en movimiento perpetuo del Token



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido (Edsger W. Dijkstra 1974)

Ofrece protección contra fallas transitorias: conducta temporal errónea de algunos procesos del sistema.

Cuando las condiciones retornan a la normalidad el proceso reasume su ejecución, pero debido a la falla temporal, el estado global podría ser arbitrario.

La estabilidad garantiza la convergencia al comportamiento requerido y a un estado global válido.

Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Hipótesis

- Comunicación Síncrona
- Topología lógica de anillo.
- En la solución se consideran N procesos, numerados del 0 a N - 1.
- Variables y constantes utilizadas:

  i = Identificador del proceso actual.

  S = Se refiere al estado del proceso i.

  L = Se refiere al estado del proceso anterior.

  K = Constante mayor o igual que N .

Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

En cada máquina el algoritmo se ejecuta en concurrencia con la aplicación. El algoritmo en ejecución consta de tres procesos relacionados, es decir, se forman tres hilos de ejecución a los que llamaremos sub – procesos:

```
Var   S,L,i : Entero

Sub – Proceso 1

        Enviar ( S )  a  (i + 1  mod  k) ;


Sub – Proceso 2

        Si (i = 0) entonces
              Recibir (L)  de  N
        Sino
              Recibir (L)  de  (i -1)
        FinSi
```

Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Sub – Proceso 3

```
Cuando (Necesita_entrar_sección_crítica) Hacer

    Si (i = 0) entonces
     Cuando ( L =S ) Hacer
                  Ir_a_sección_crítica();

        S = (L + 1)  mod  K;
     FinCuando

   Sino
     Cuando ( L <> S )  Hacer
          Ir_a_sección_crítica();
                S = L   ;

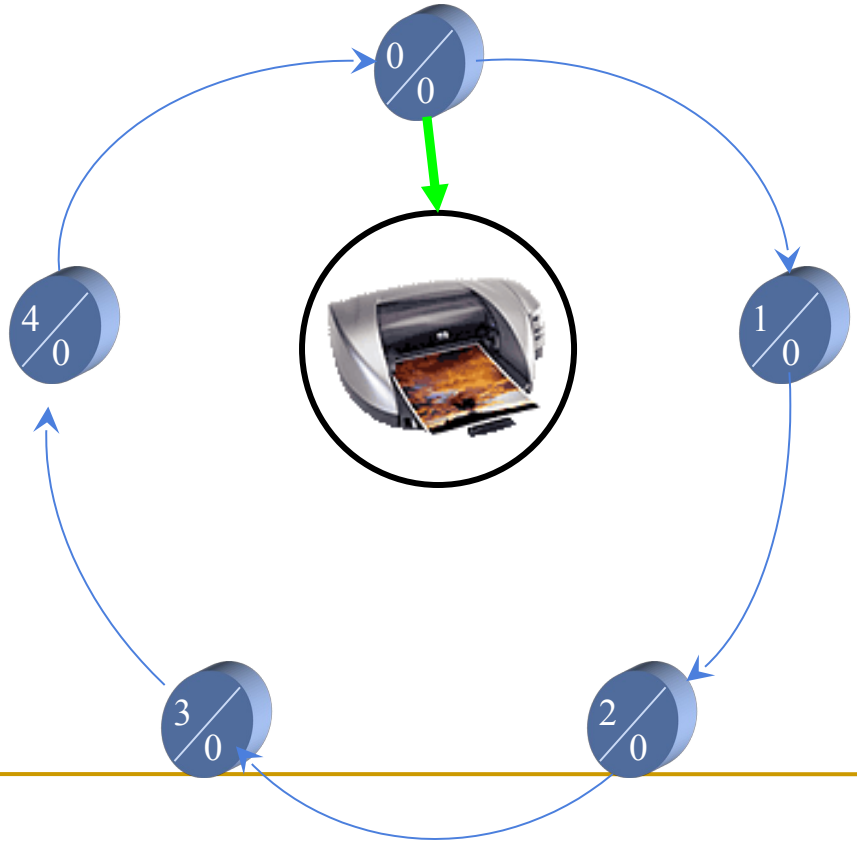
     FinCuando

    FinSi

FinCuando
```

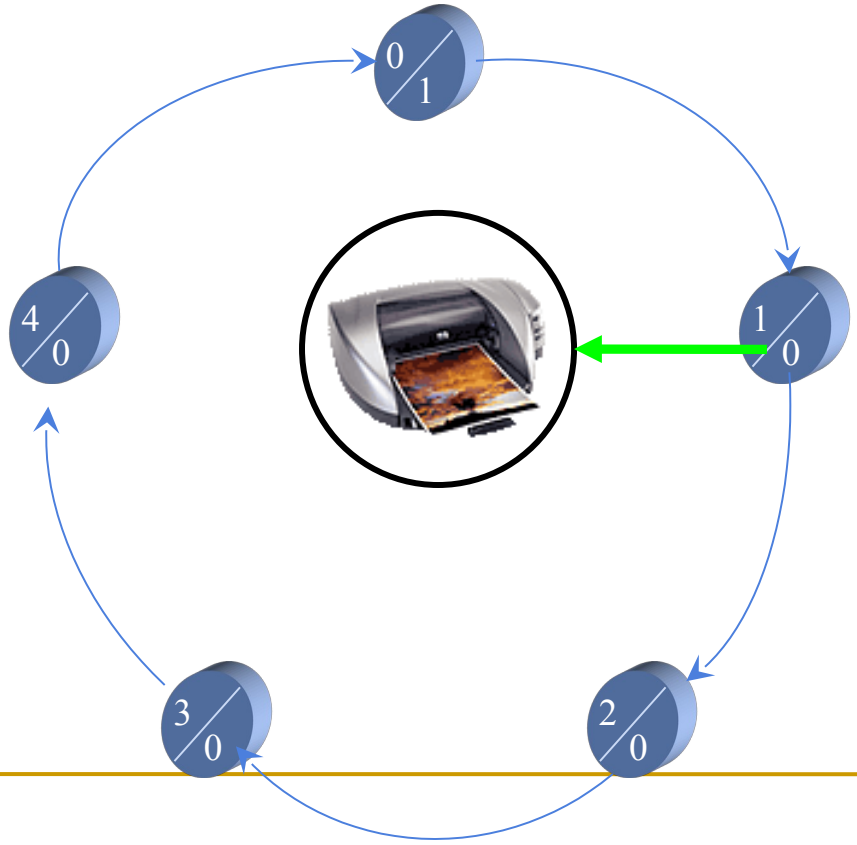
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



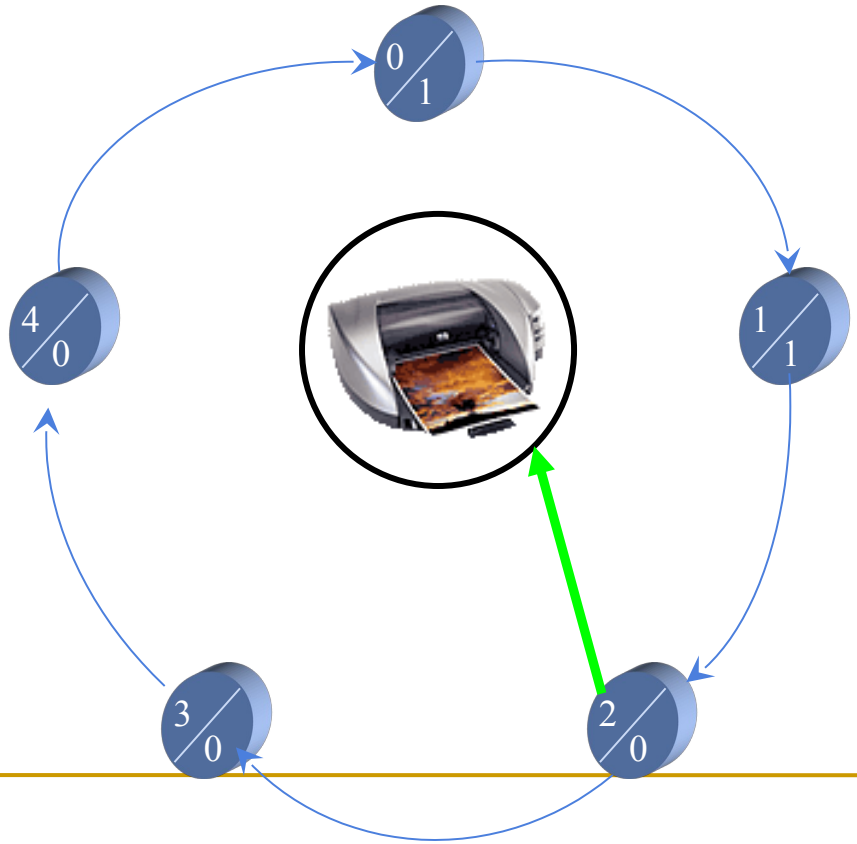
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



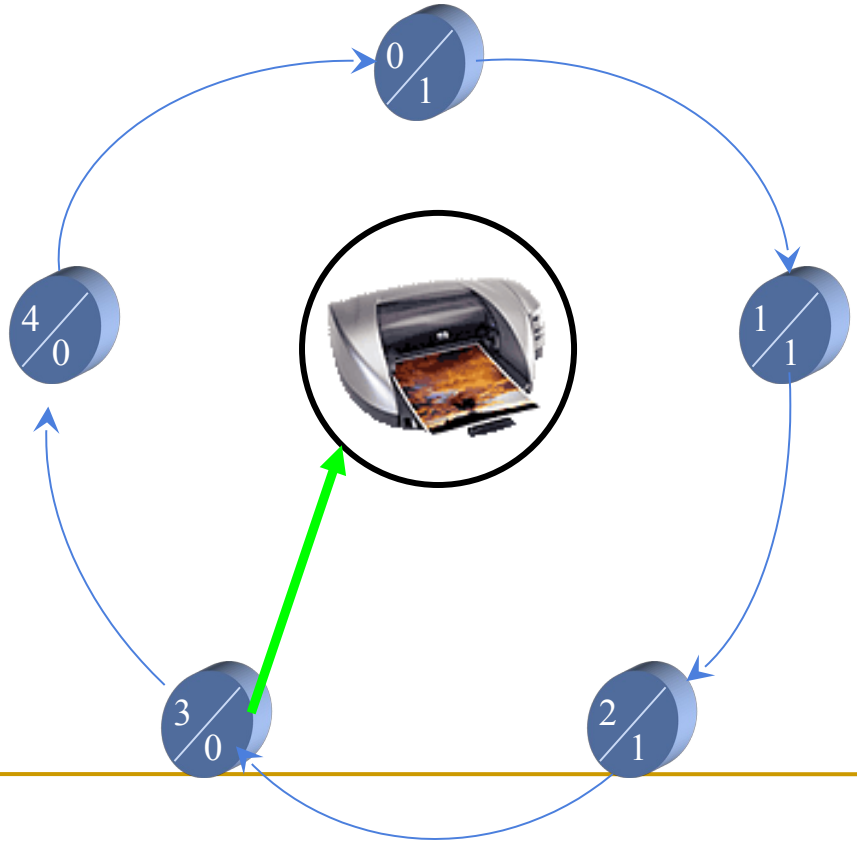
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



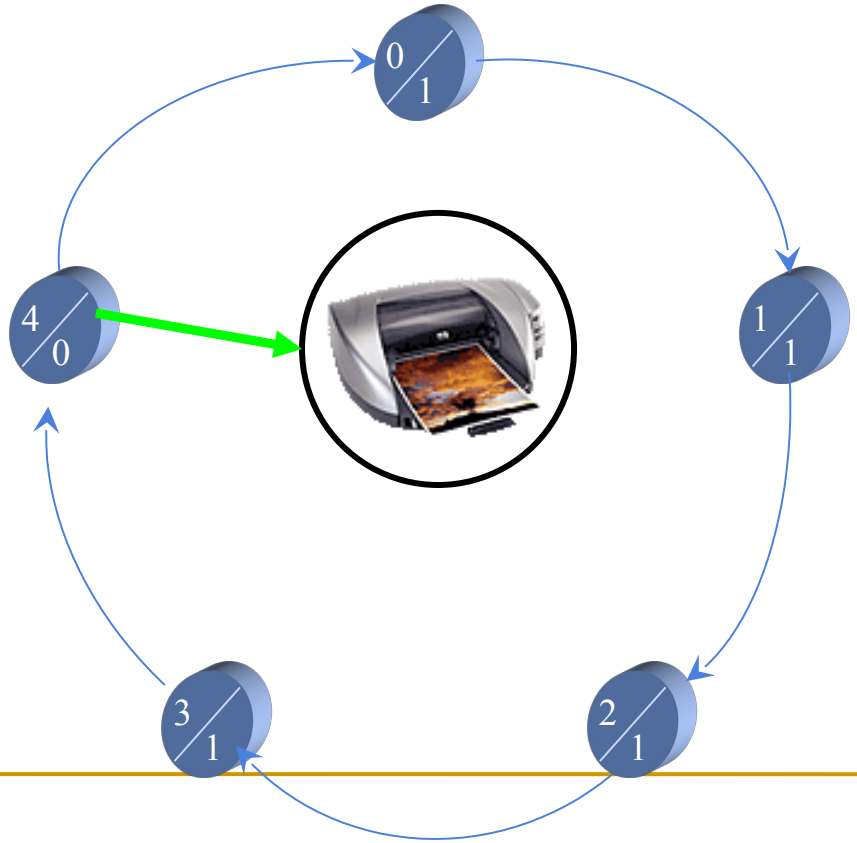
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



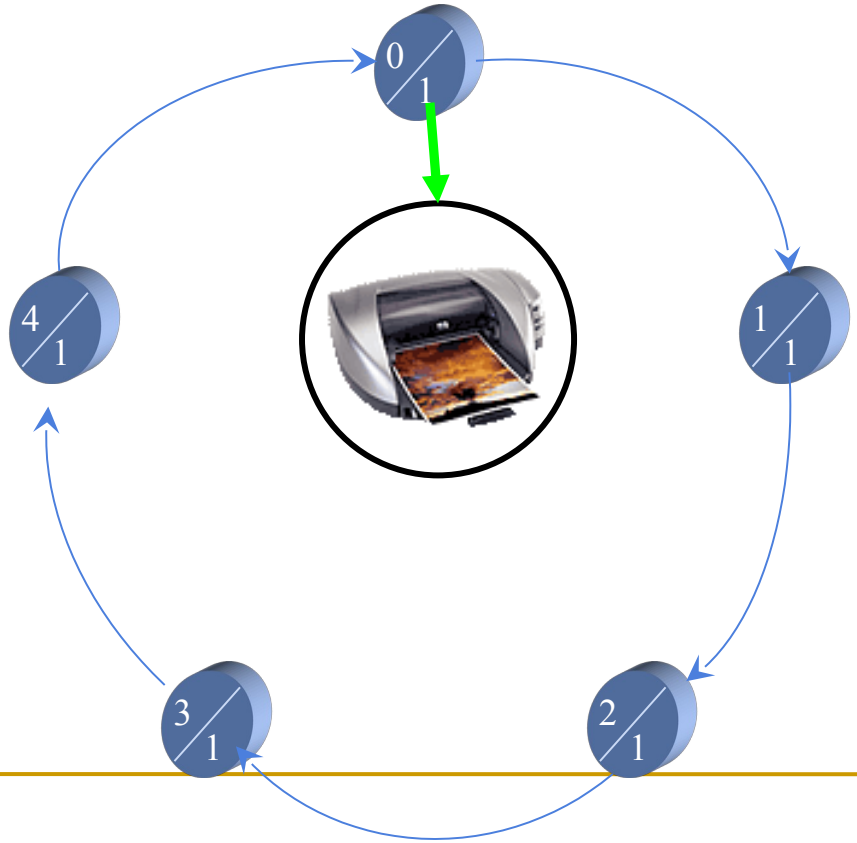
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



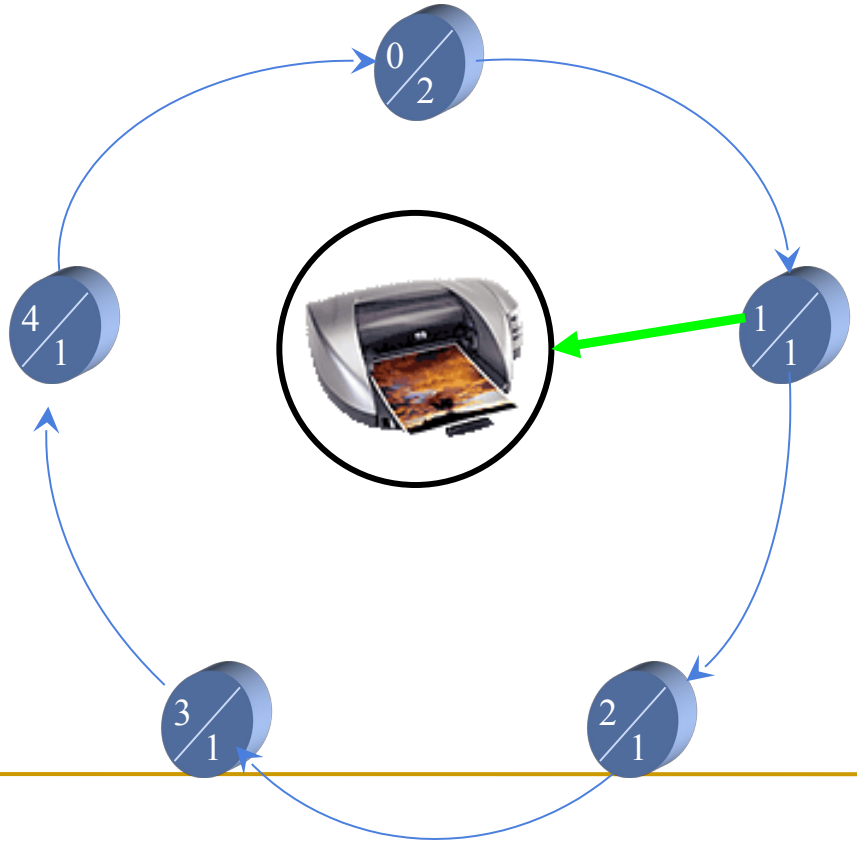
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



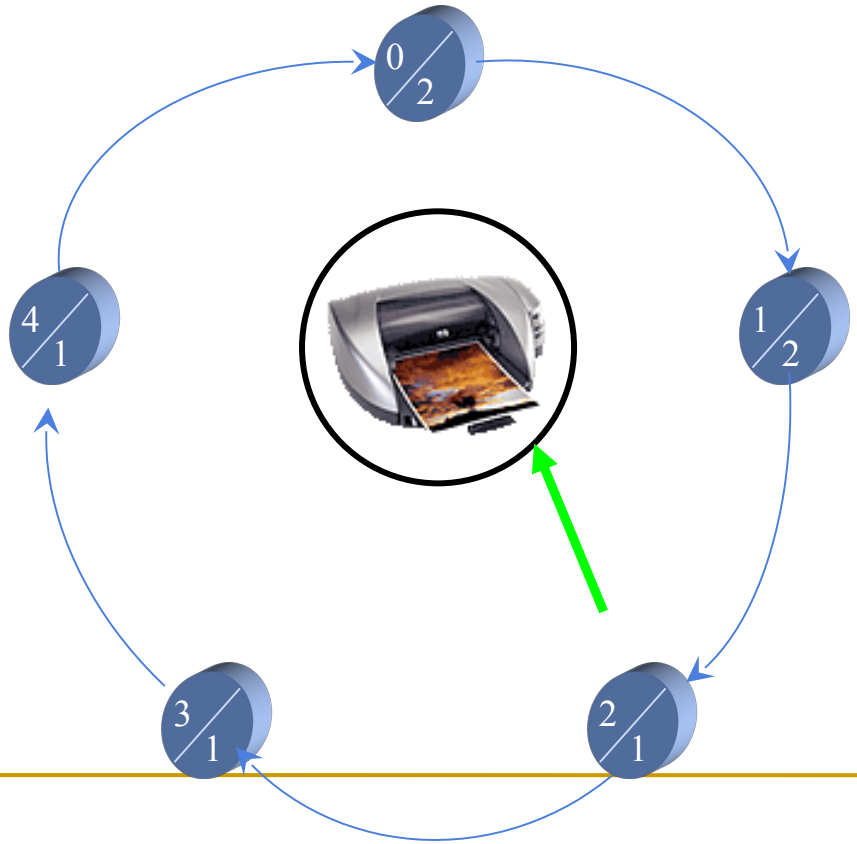
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



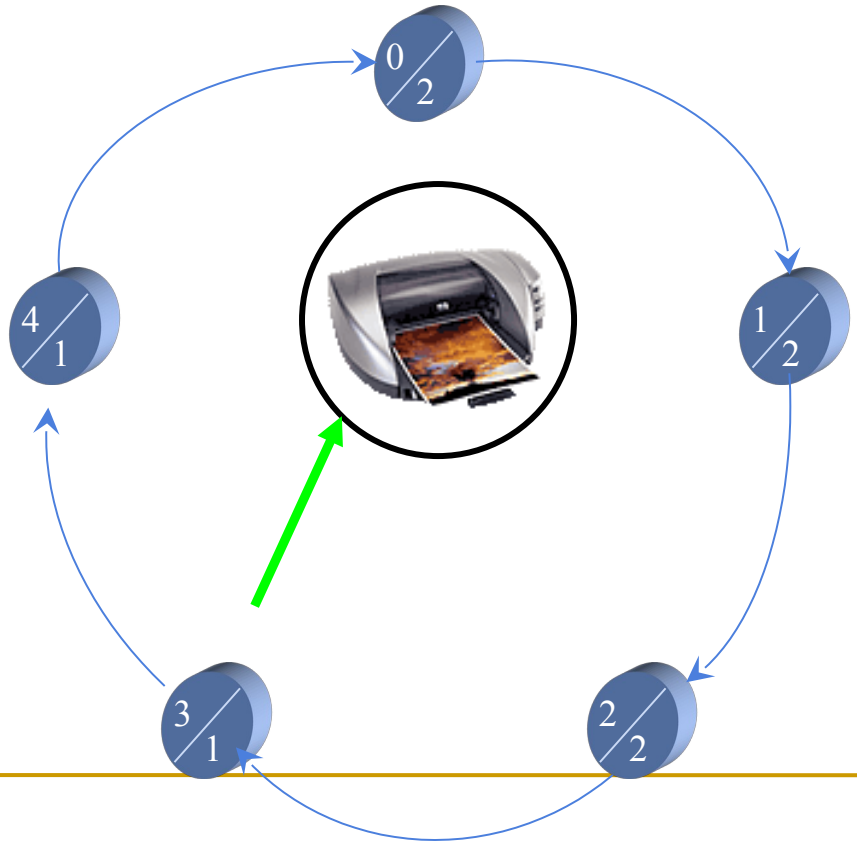
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



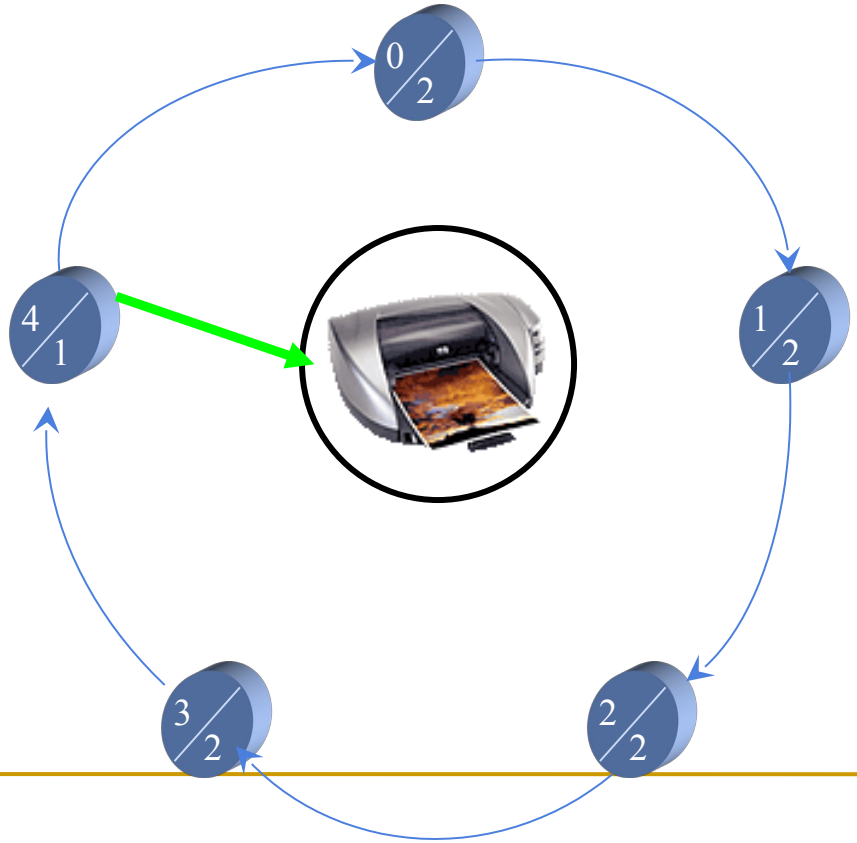
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



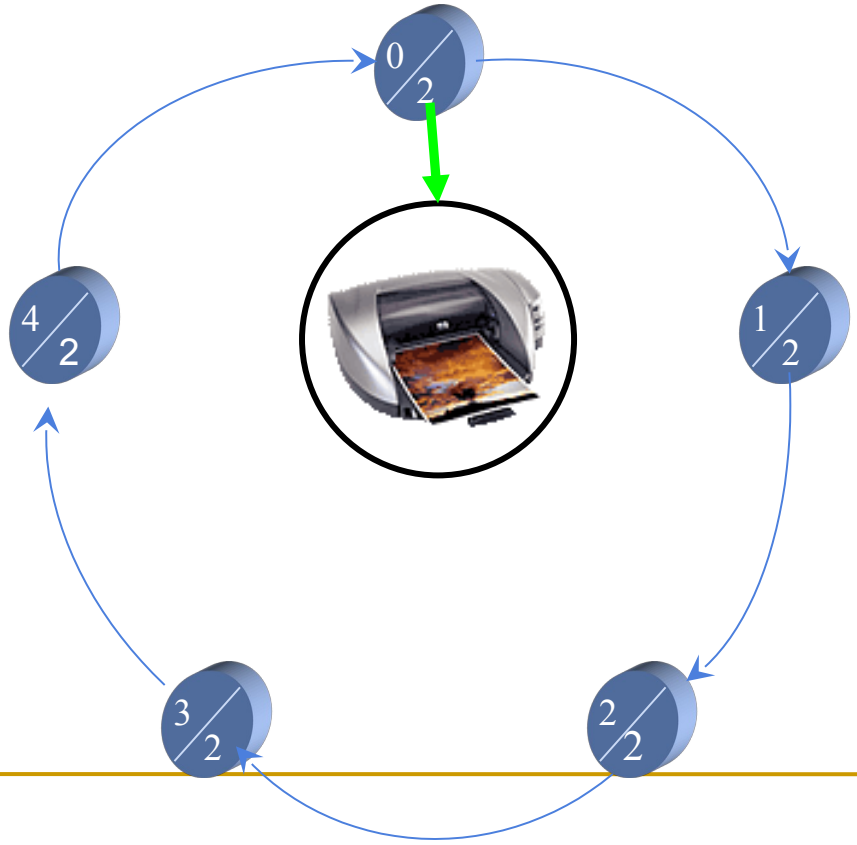
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



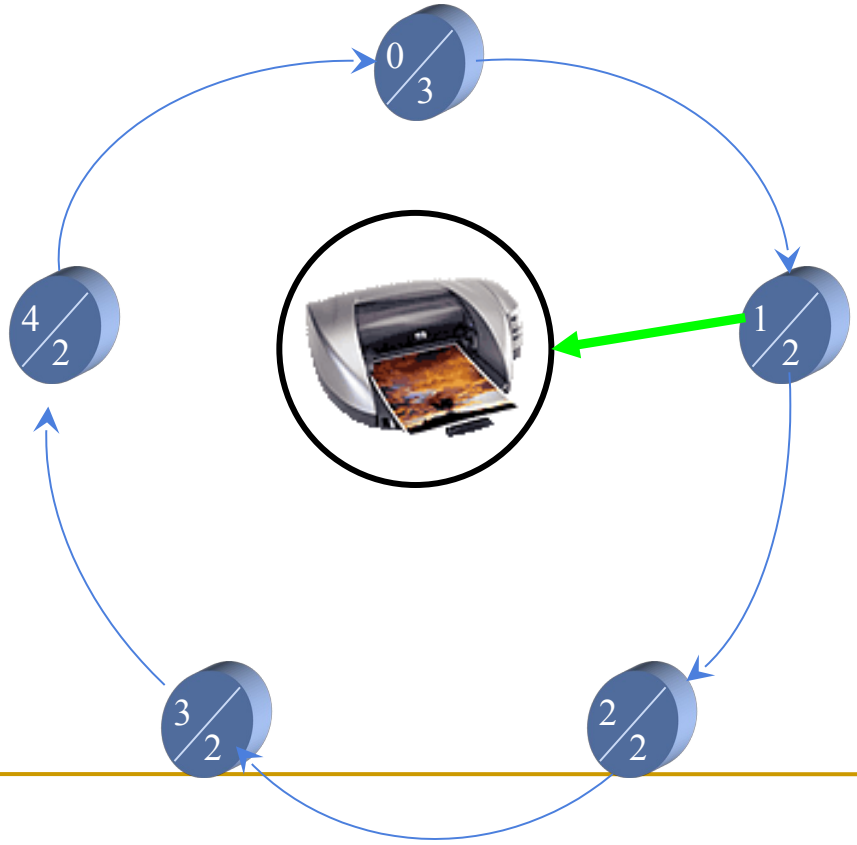
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



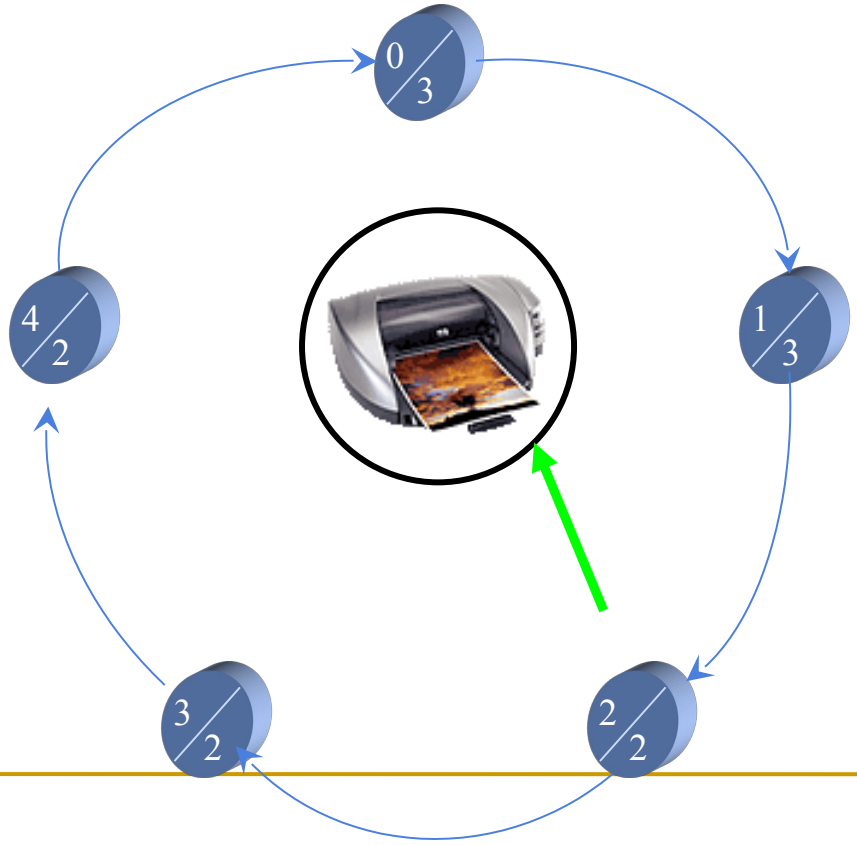
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



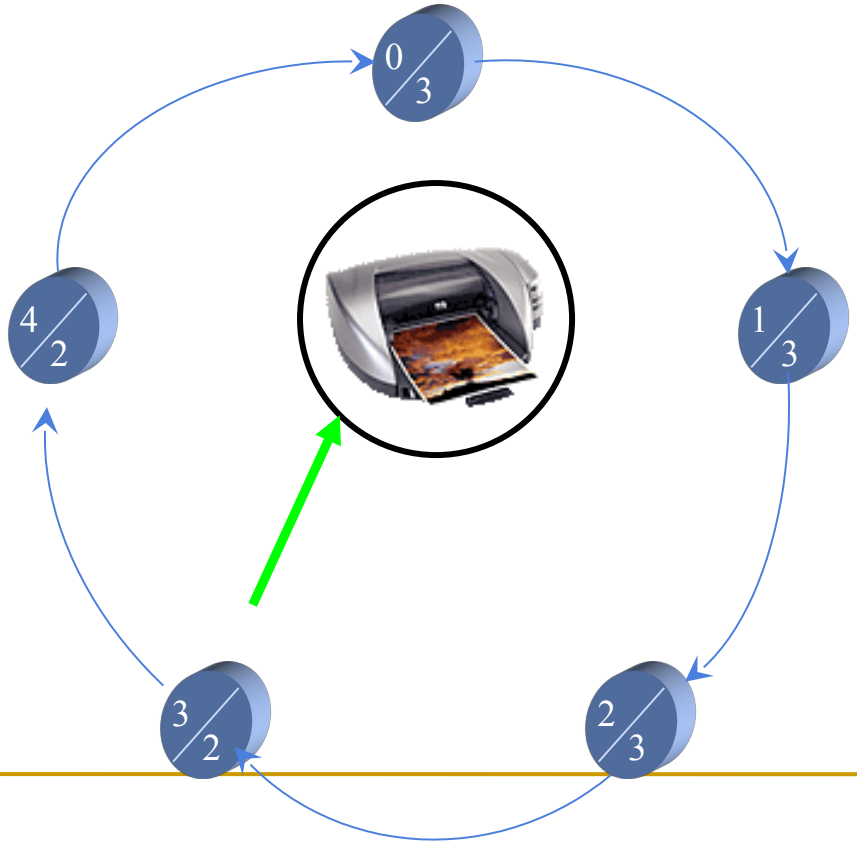
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



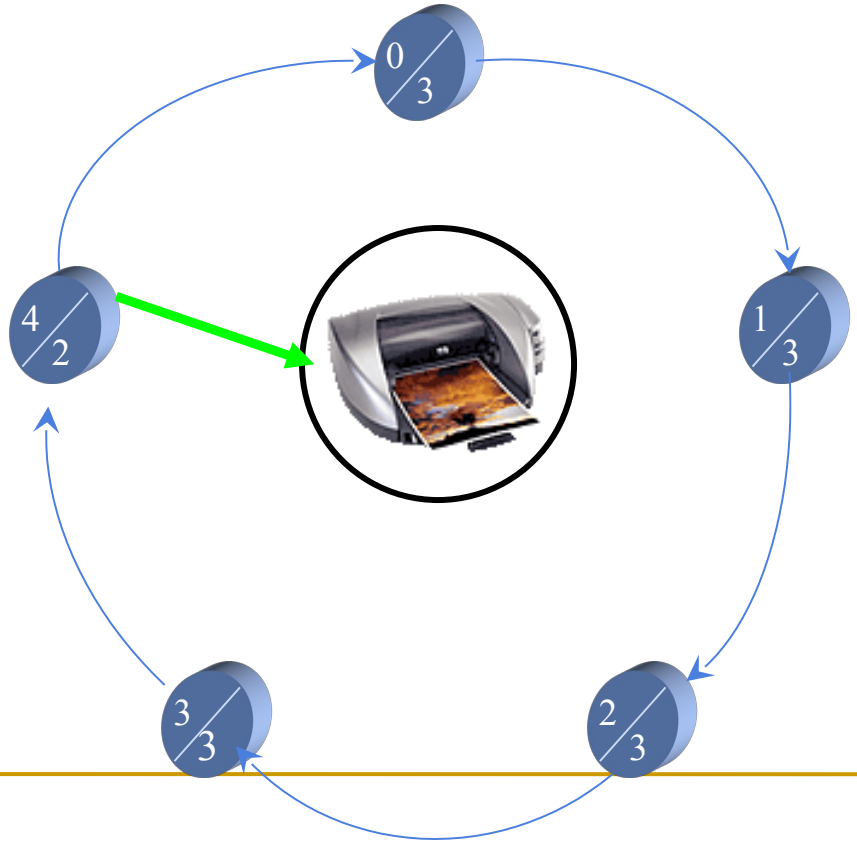
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



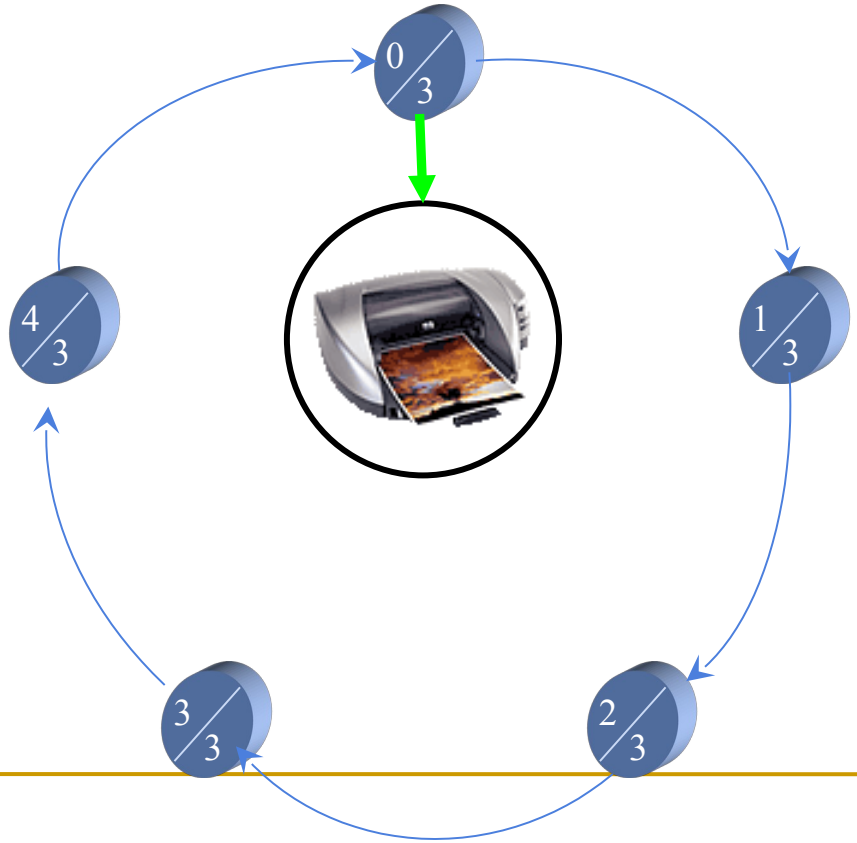
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



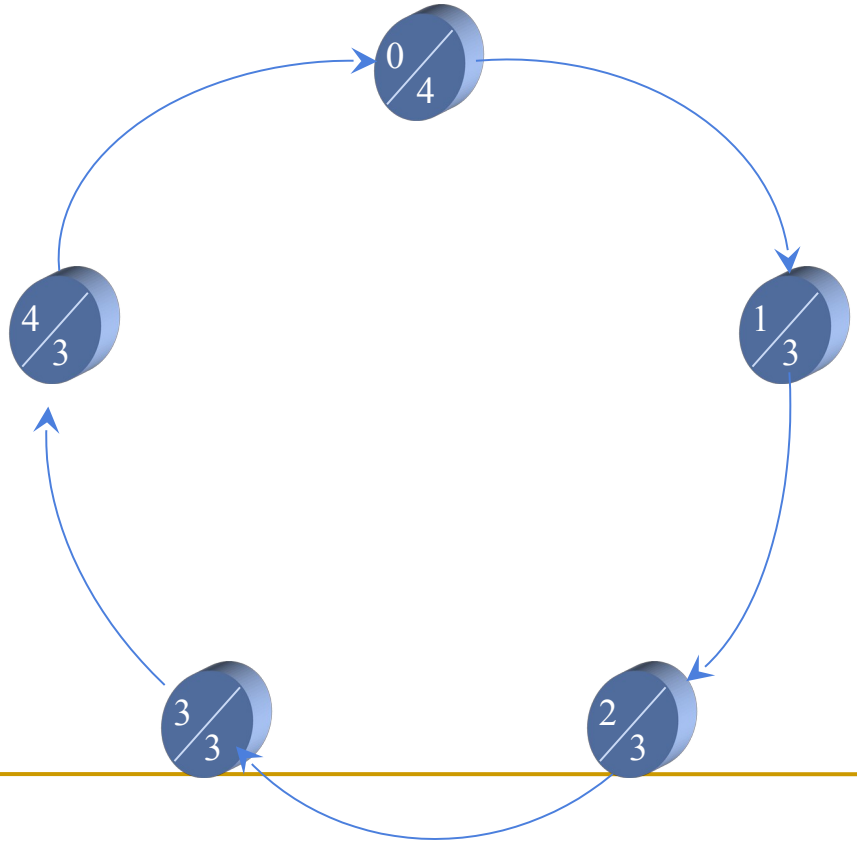
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

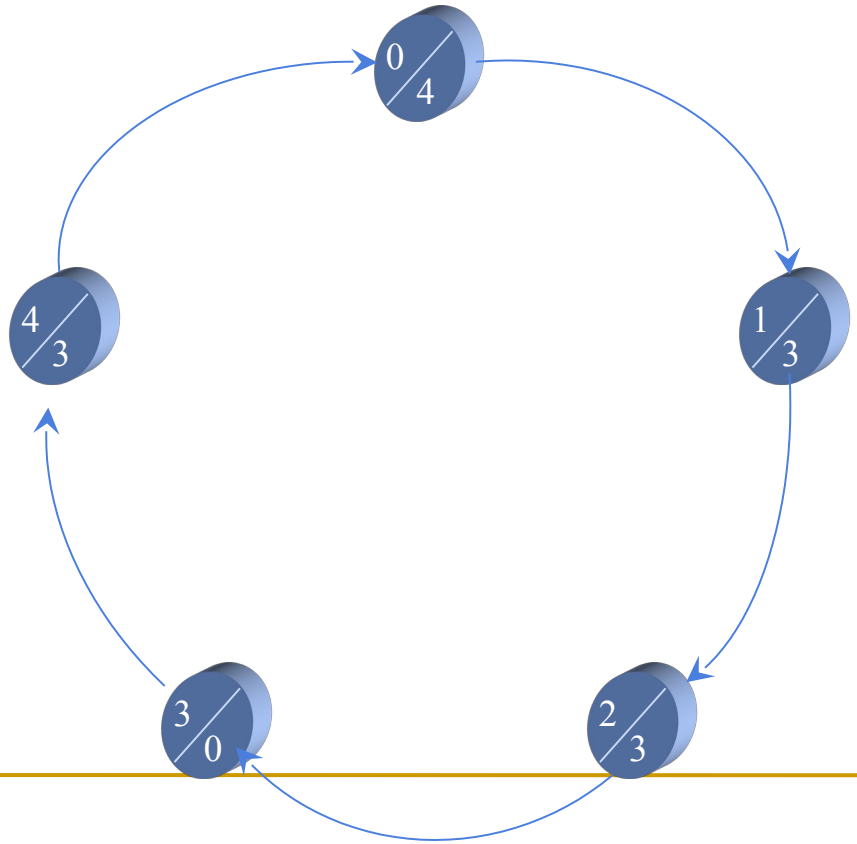
Solución para N = 5 y K = 6



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

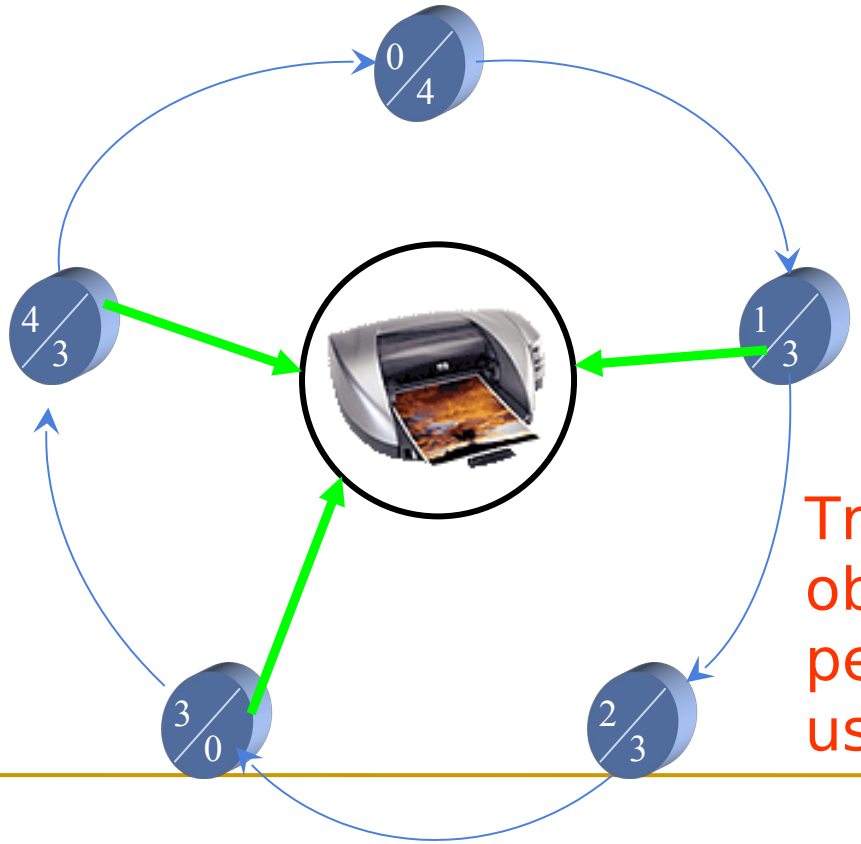
Ocurre una falla



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

Ocurre una falla

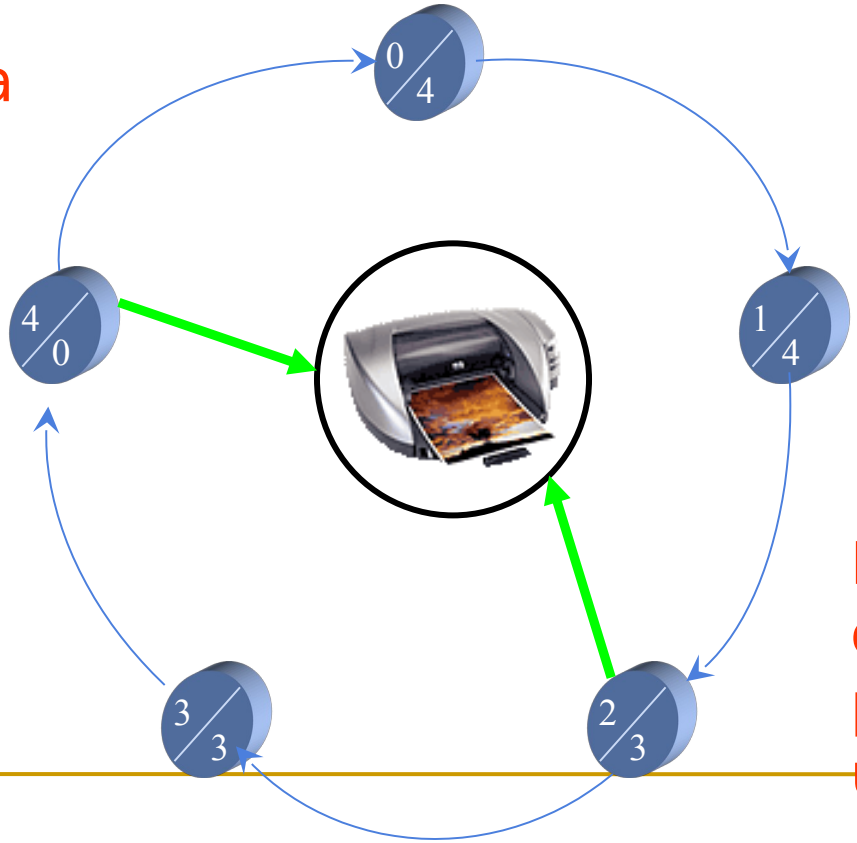


Tres procesos obtienen permiso para usar el recurso

Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

Ocurre una falla

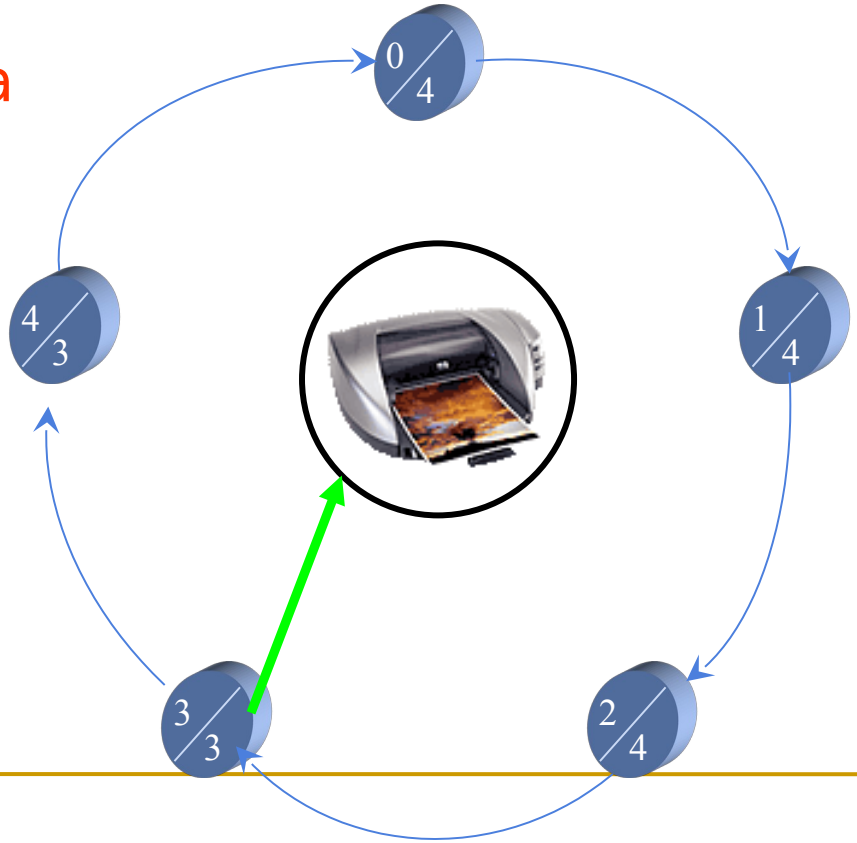


Dos procesos obtienen permiso para usar el recurso

Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

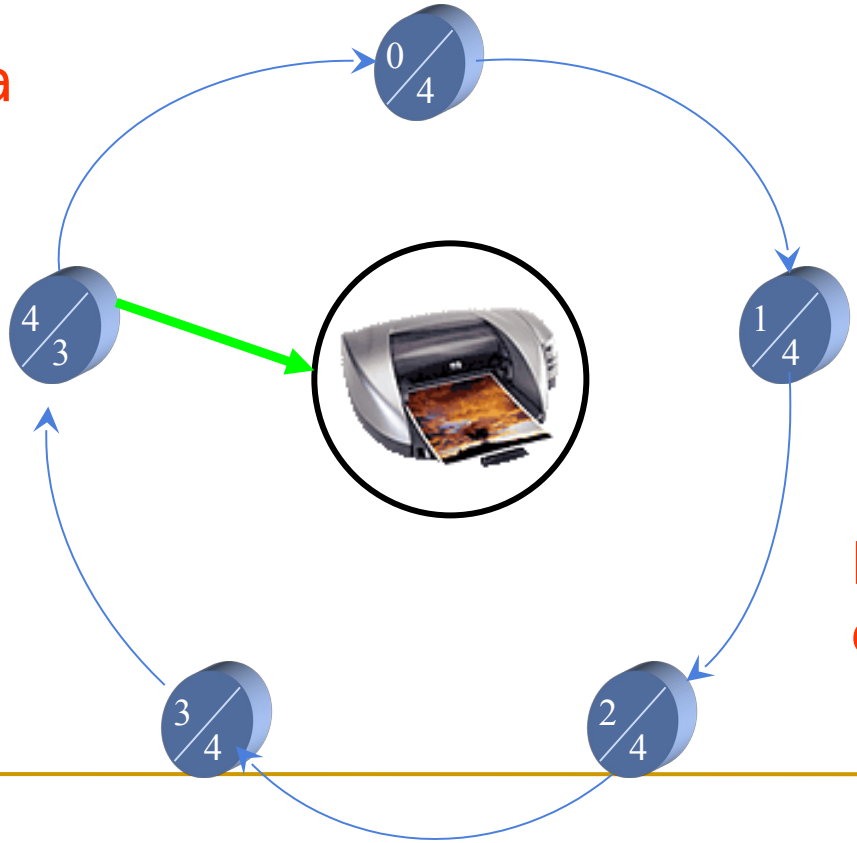
Ocurre una falla



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

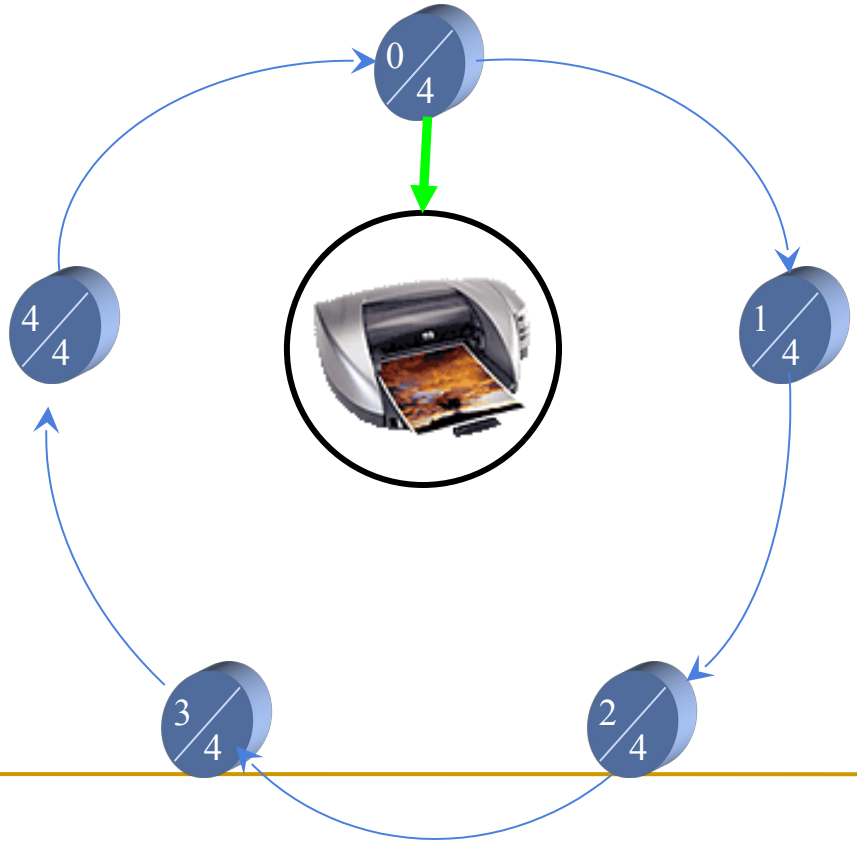
Ocurre una falla



El algoritmo se estabiliza

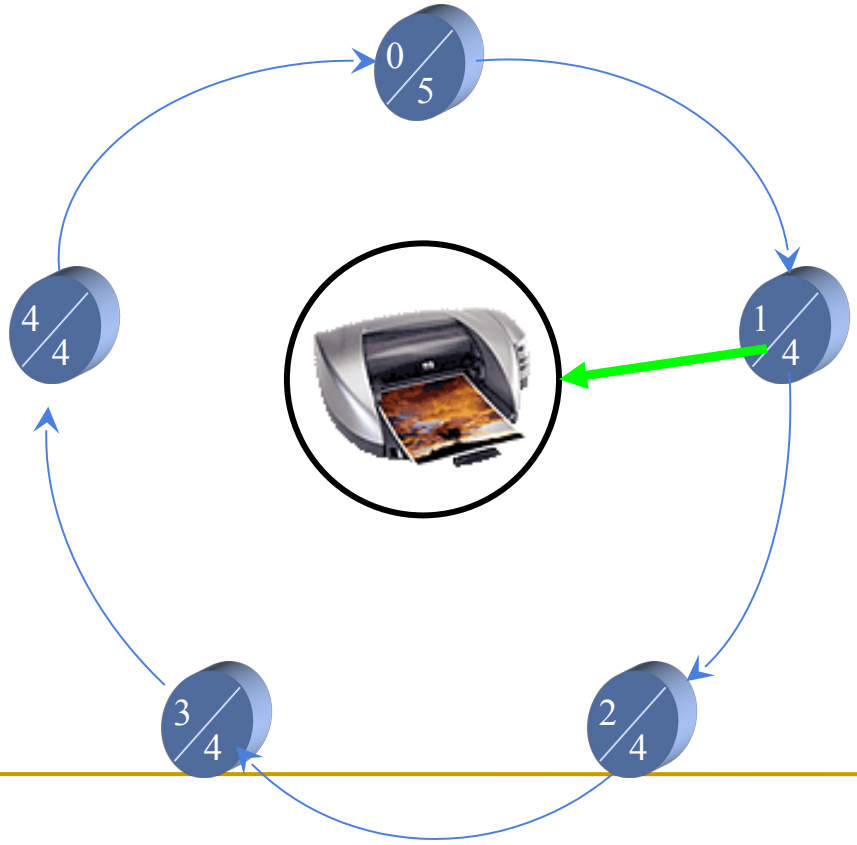
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



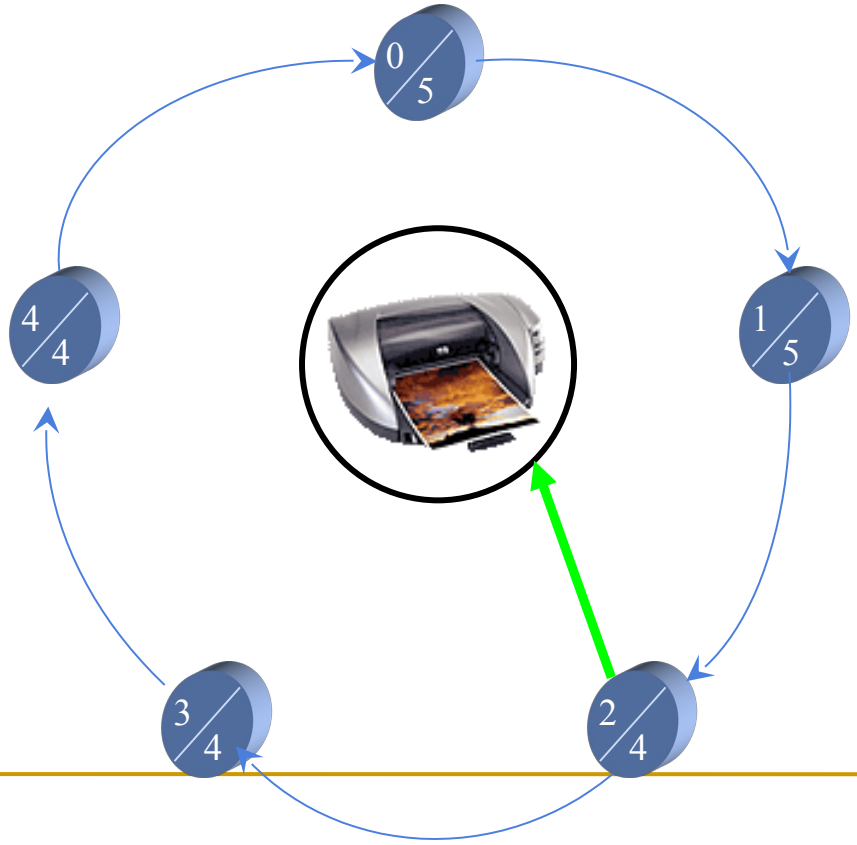
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



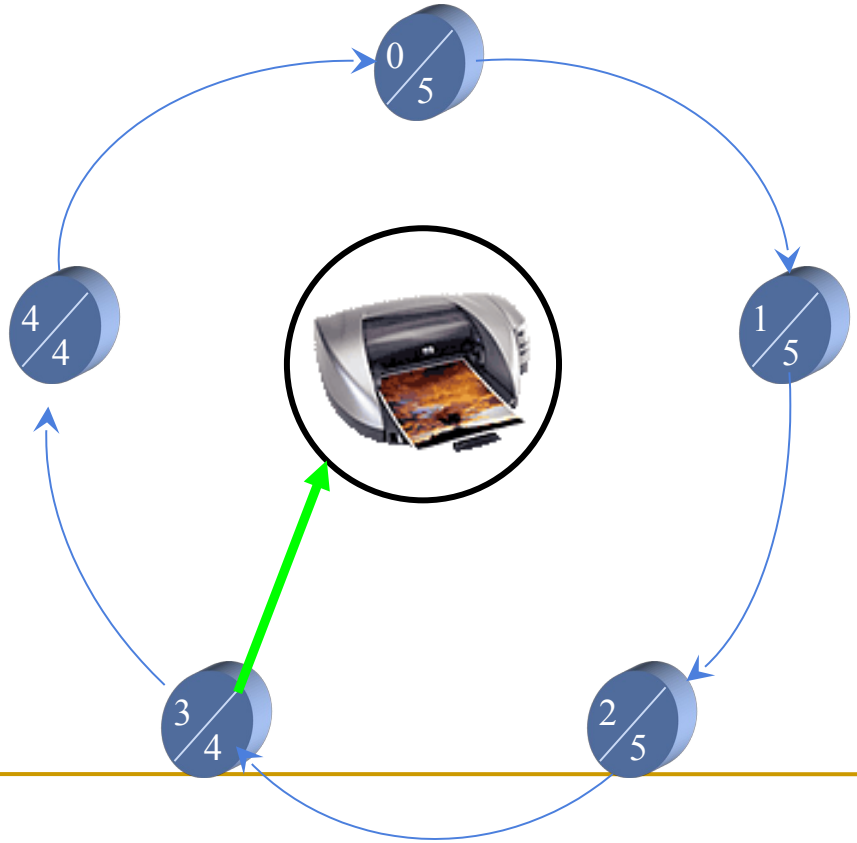
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



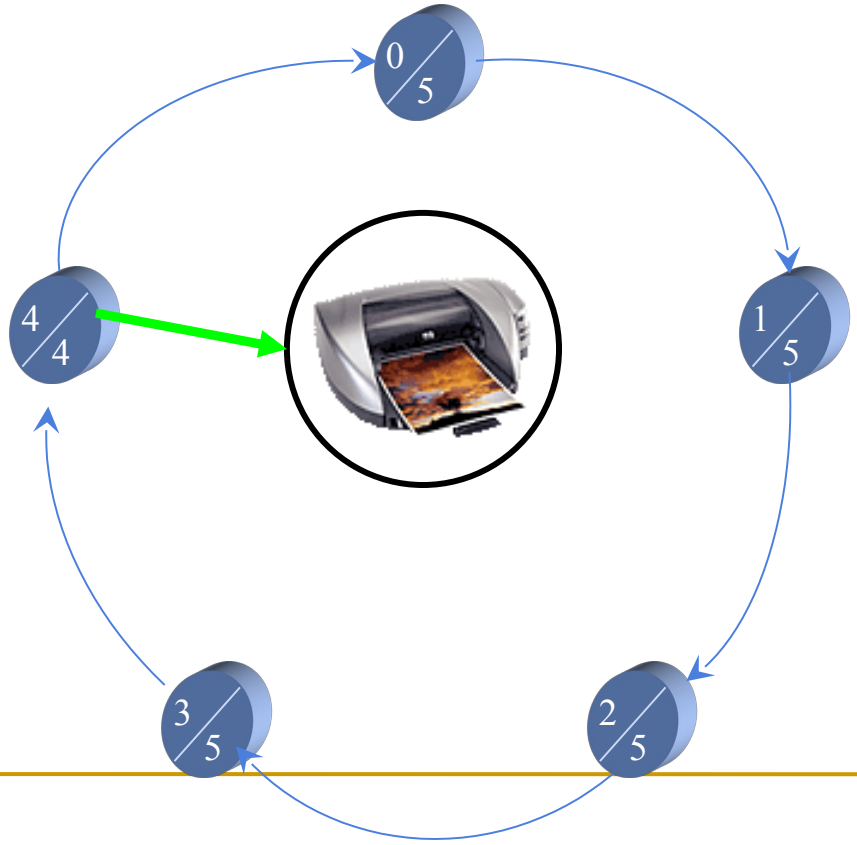
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



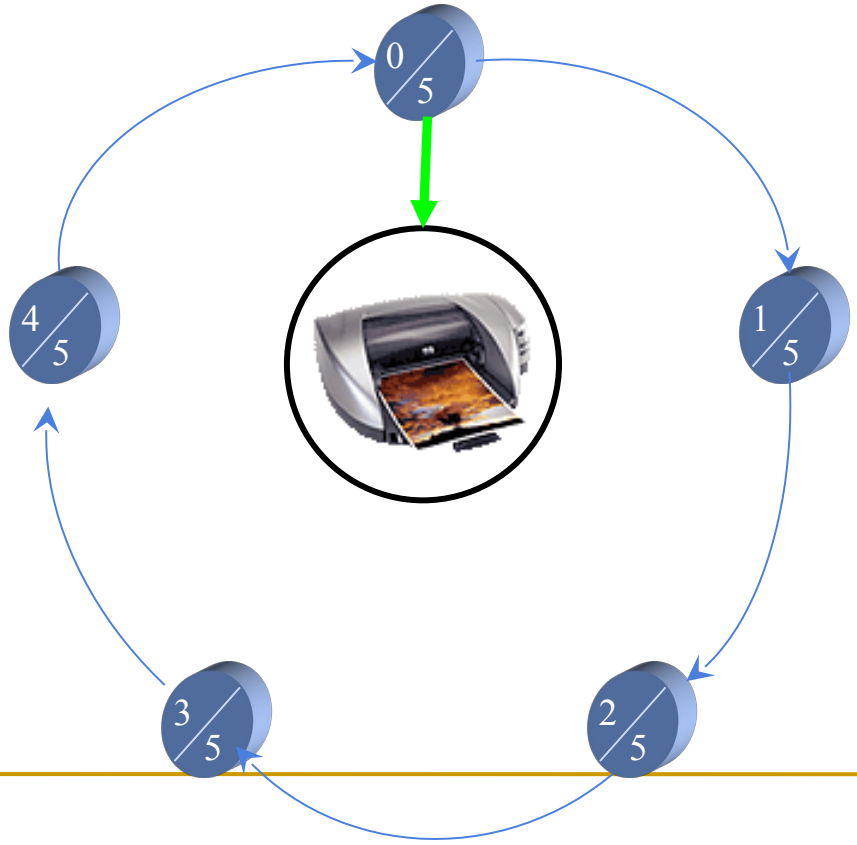
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



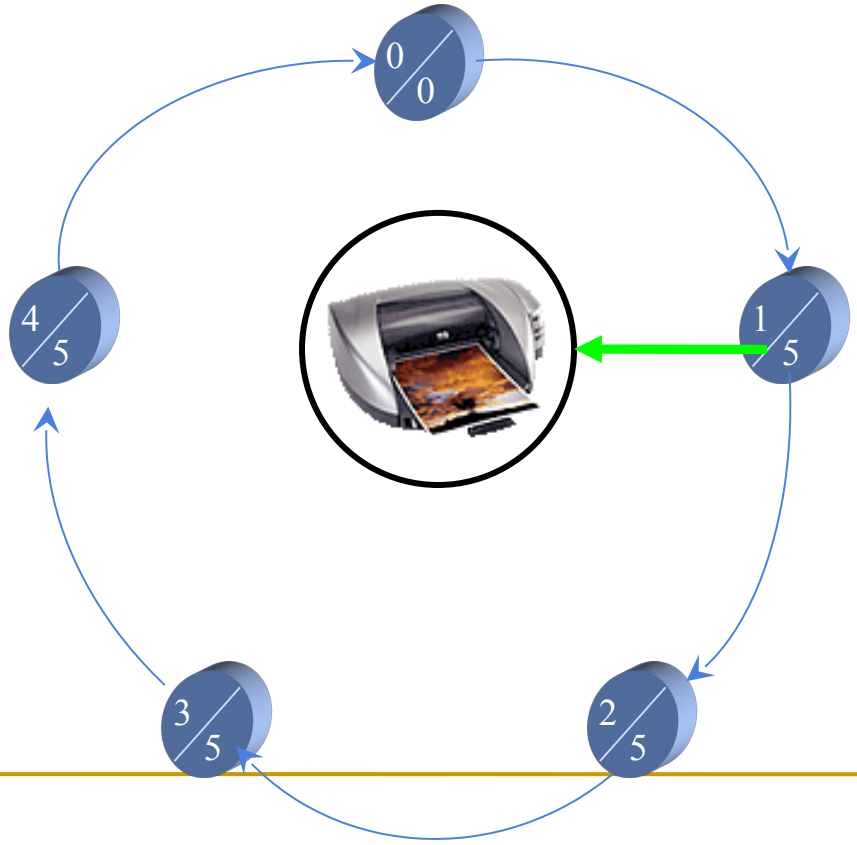
Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6



Algoritmo Estable para la Exclusión Mutua en un Sistema Distribuido

Solución para N = 5 y K = 6

